

Categorical Cryptography: An axiomatic theory of simulation-based cryptography

Proving cryptographic properties about real-world programs

Andre Knispel

September 2026

IOHK

In search of a language

Simulation-based cryptography — security as indistinguishability from an ideal specification — is the right way to reason about large cryptographic systems.

Simulation-based cryptography — security as indistinguishability from an ideal specification — is the right way to reason about large cryptographic systems.

But it lives in *models*, not in a *language*: operational machine models full of bookkeeping, which nobody uses in practice. There are also lots of variants, raising all sorts of compatibility questions.

This talk in one sentence

Simulation-based cryptography — security as indistinguishability from an ideal specification — is the right way to reason about large cryptographic systems.

But it lives in *models*, not in a *language*: operational machine models full of bookkeeping, which nobody uses in practice. There are also lots of variants, raising all sorts of compatibility questions.

The goal of this work:

find the mathematical language of simulation-based security
and prove theorems in & metatheorems about it.

Example: a chimeric UTxO ledger

A toy ledger that implements a tiny amount of the accounting features of Cardano. State: $\text{utxo} : \text{TxIn} \rightarrow \text{TxOut}$ and $\text{accts} : \text{Addr} \rightarrow \mathbb{N}$; a transaction $tx = (\text{ins}, \text{wdrls}, \text{outs})$ spends UTxOs *and* withdraws from accounts; $\text{outsOf } tx$ stores outs at the keys $(\text{hash } tx, i)$:

$$\frac{\begin{array}{l} \text{ins} \subseteq \text{dom utxo} \qquad \text{wdrls} \leq \text{accts} \\ \text{balance } (\text{utxo} \mid \text{ins}) + \sum \text{wdrls} = \text{balance outs} \end{array}}{(\text{utxo}, \text{accts}) \xrightarrow{tx} ((\text{utxo} \mid \text{ins}^c) \cup \text{outsOf } tx, \text{accts} - \text{wdrls})}$$

Example: a chimeric UTxO ledger

A toy ledger that implements a tiny amount of the accounting features of Cardano. State: $\text{utxo} : \text{TxIn} \rightarrow \text{TxOut}$ and $\text{accts} : \text{Addr} \rightarrow \mathbb{N}$; a transaction $tx = (\text{ins}, \text{wdrls}, \text{outs})$ spends UTxOs *and* withdraws from accounts; $\text{outsOf } tx$ stores outs at the keys $(\text{hash } tx, i)$:

$$\frac{\begin{array}{l} \text{ins} \subseteq \text{dom utxo} \qquad \text{wdrls} \leq \text{accts} \\ \text{balance } (\text{utxo} \mid \text{ins}) + \sum \text{wdrls} = \text{balance outs} \end{array}}{(\text{utxo}, \text{accts}) \xrightarrow{tx} ((\text{utxo} \mid \text{ins}^c) \cup \text{outsOf } tx, \text{accts} - \text{wdrls})}$$

Preservation of value: if $s_0 \xrightarrow{tx_1} s_1 \xrightarrow{tx_2} \cdots \xrightarrow{tx_n} s_n$ and $\text{hash } tx_1, \dots, \text{hash } tx_n$ are pairwise distinct and none of them occurs in s_0 , then

$$\text{total } s_n = \text{total } s_0, \quad \text{where} \quad \text{total } (\text{utxo}, \text{accts}) = \text{balance utxo} + \sum \text{accts}.$$

We can replay some transactions, violating POV

Take $tx = (\emptyset, \text{wdrls}, \text{outs})$ — no inputs, funded entirely from an account.

- **apply it once**: legal; outputs appear at $(\text{hash } tx, i)$, the account is debited
- **apply it again**: still legal — with $\text{ins} = \emptyset$, nothing prevents the same transaction from stepping twice
- but the outputs $(\text{hash } tx, i)$ already exist: the union adds nothing, while the account is debited again

We can replay some transactions, violating POV

Take $tx = (\emptyset, \text{wdrls}, \text{outs})$ — no inputs, funded entirely from an account.

- **apply it once**: legal; outputs appear at $(\text{hash } tx, i)$, the account is debited
- **apply it again**: still legal — with $\text{ins} = \emptyset$, nothing prevents the same transaction from stepping twice
- but the outputs $(\text{hash } tx, i)$ already exist: the union adds nothing, while the account is debited again

The preservation of value theorem doesn't cover this case, because the precondition isn't satisfied.

We can replay some transactions, violating POV

Take $tx = (\emptyset, \text{wdrls}, \text{outs})$ — no inputs, funded entirely from an account.

- **apply it once**: legal; outputs appear at $(\text{hash } tx, i)$, the account is debited
- **apply it again**: still legal — with $\text{ins} = \emptyset$, nothing prevents the same transaction from stepping twice
- but the outputs $(\text{hash } tx, i)$ already exist: the union adds nothing, while the account is debited again

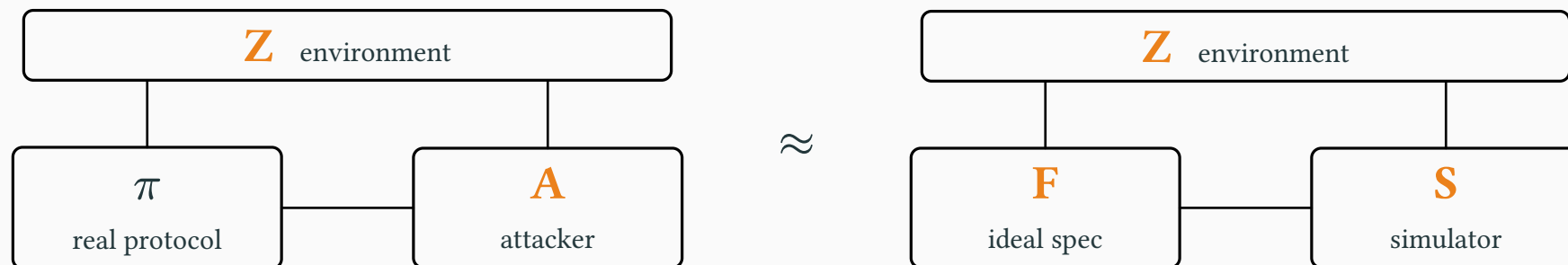
The preservation of value theorem doesn't cover this case, because the precondition isn't satisfied.

To catch this, we need to somehow eliminate the faulty precondition in the statement. Properly modelling cryptography is one way to do that.

Universal Composability (Canetti 2001)

Let's back up a bit and talk about UC/simulation-based crypto.

A protocol is secure if it is **indistinguishable from its specification**:



For every attacker on the real protocol there is a “simulated attacker” on the ideal spec that **no environment can tell apart**.

The composition theorem

Write $\pi \underset{\text{UC}}{\leq} F$ for “ π emulates **F**”. Then composition is **monotone** with respect to $\underset{\text{UC}}{\leq}$:

$$\pi \underset{\text{UC}}{\leq} F \implies \rho \circ \pi \underset{\text{UC}}{\leq} \rho \circ F$$

In our earlier example, ρ would be the ledger, and π and F the real hash function and random oracle functionality respectively.

The composition theorem

Write $\pi \underset{\text{UC}}{\leq} F$ for “ π emulates **F**”. Then composition is **monotone** with respect to $\underset{\text{UC}}{\leq}$:

$$\pi \underset{\text{UC}}{\leq} F \implies \rho \circ \pi \underset{\text{UC}}{\leq} \rho \circ F$$

In our earlier example, ρ would be the ledger, and π and F the real hash function and random oracle functionality respectively.

- security proofs are **modular**
- design against clean specs, implement piece by piece
- all properties that environments can test are preserved

It's not so easy...

UC in the crypto literature:

- interactive Turing machines, activation tokens, session & party ids, control functions, runtime-budget bookkeeping...
- **almost 100 pages, patched for 20 years** — plus a whole family of competing models keeping the idea and re-engineering the details: GNUC, IITM, iUC, SUC, Constructive Cryptography / Abstract Cryptography
- and because that's not enough, some crypto papers do their own minor one-off variants

It's not so easy...

UC in the crypto literature:

- interactive Turing machines, activation tokens, session & party ids, control functions, runtime-budget bookkeeping...
- **almost 100 pages, patched for 20 years** — plus a whole family of competing models keeping the idea and re-engineering the details: GNUC, IITM, iUC, SUC, Constructive Cryptography / Abstract Cryptography
- and because that's not enough, some crypto papers do their own minor one-off variants

The concepts are clearly robust, but the language is not.

Claim: there is a much simpler abstract theory that handles a wide range of models and variants and has a well-behaved metatheory

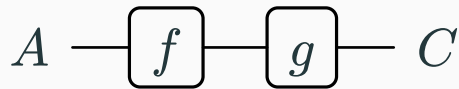
The categorical skeleton

Boxes and wires

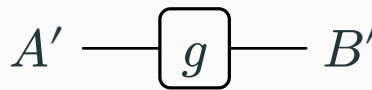
A **category** (for today): interfaces and machines.



a machine
with interfaces



composition $g \circ f$:
plug them together



tensor $f \otimes g$:
run side by side

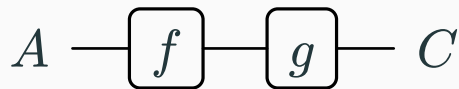
plus neutral elements and a bunch of laws.

Boxes and wires

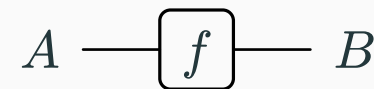
A **category** (for today): interfaces and machines.



a machine
with interfaces



composition $g \circ f$:
plug them together



tensor $f \otimes g$:
run side by side

plus neutral elements and a bunch of laws.

Equations between diagrams say: **only connectivity matters**.

Our boxes: stateful, effectful, interactive functions

The theory works for any base category with some extra structure (a UC-setup), but this is the key example.

An object is a **channel**: a pair of types, for the messages flowing each way. A morphism $A \rightarrow B$ is a machine you can **converse** with:



- **interactive**: each incoming message may emit a message on either side
- **stateful**: steps read and update the private state σ
- **effectful**: a step may have effects in a fixed monad, typically randomness

Our boxes: stateful, effectful, interactive functions

The theory works for any base category with some extra structure (a UC-setup), but this is the key example.

An object is a **channel**: a pair of types, for the messages flowing each way. A morphism $A \rightarrow B$ is a machine you can **converse** with:

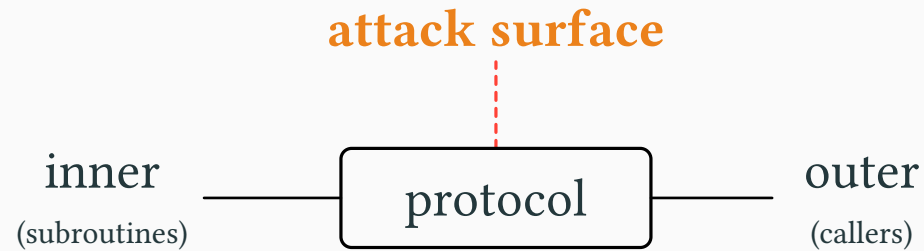


- **interactive**: each incoming message may emit a message on either side
- **stateful**: steps read and update the private state σ
- **effectful**: a step may have effects in a fixed monad, typically randomness

Think of this as our replacement for Turing machines. We could also use the subcategory of f 's that are poly time, etc.

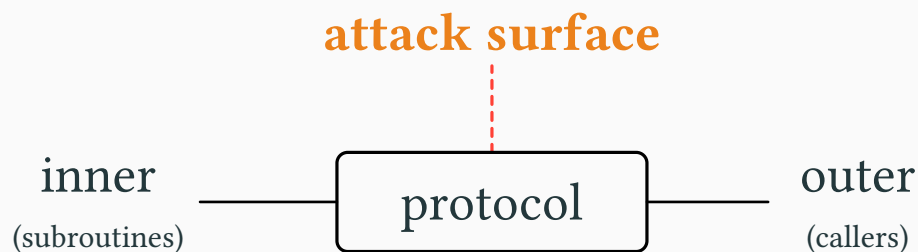
Ingredient 2: attacks are extra effects

An **attack** on a protocol is typically an extra open interface glued onto it:



Ingredient 2: attacks are extra effects

An **attack** on a protocol is typically an extra open interface glued onto it:



- a protocol with an attack surface is a morphism $A \rightarrow \mathcal{M}_X B$
(the “grade” records *which* surface; composing attacks composes grades)
- the standard example is $\mathcal{M} = \otimes$, but varying this gives honest-but-curious attackers, multiple non-cooperating attackers, etc.
- protocol + attack = one morphism in the **Kleisli category** of $\mathcal{M}$

Ingredient 3: distinguishing

Who decides whether two systems are “the same”?

- fix a family $\mathcal{E}$ of **tests** (environments) — formally, a presheaf on our base category
- the key feature of this family (and what makes it a presheaf) is that we can compose a test with a protocol to obtain a new test
- $f \approx_{\mathcal{E}} g$ iff every test E composed with f and g gives the same test: $\mathcal{E}(f)E = \mathcal{E}(g)E$

Ingredient 3: distinguishing

Who decides whether two systems are “the same”?

- fix a family $\mathcal{E}$ of **tests** (environments) — formally, a presheaf on our base category
- the key feature of this family (and what makes it a presheaf) is that we can compose a test with a protocol to obtain a new test
- $f \approx_{\mathcal{E}} g$ iff every test E composed with f and g gives the same test: $\mathcal{E}(f)E = \mathcal{E}(g)E$

Some examples we could use, for varying notions of security:

- exact / information-theoretic: all morphisms in our base category
- computational: feasible morphisms
- indistinguishable: sequences of probability distributions, up to vanishing total-variation distance

$$\rho \leq_{\overline{\text{UC}}} \varphi \iff \text{for every attack } A \text{ on } \rho \text{ there is an attack } S \text{ on } \varphi \text{ with } A \circ \rho \approx_{\mathcal{E}} S \circ \varphi$$

UC-emulation, in one line

$$\rho \underset{\text{UC}}{\leq} \varphi \iff \text{for every attack } A \text{ on } \rho \text{ there is an attack } S \text{ on } \varphi \text{ with } A \circ \rho \underset{\mathcal{E}}{\approx} S \circ \varphi$$

The whole framework is **four ingredients**:

ingredient	formally	plays the role of
computation	a monoidal category $\mathcal{C}$	machines & channels
adversarial computation	a monoidal category $\mathcal{I}$	as above
adversary	a $\mathcal{I}$ -graded monad $\mathcal{M}$	attack surfaces
observation	a presheaf $\mathcal{E}$ on $\text{Kl}(\mathcal{M})$	environments / tests

The tuple $(\mathcal{C}, \mathcal{I}, \mathcal{M}, \mathcal{E})$ is called a UC-setup.

The metatheory falls out

For any UC-setup, we prove:

$\leq_{\text{UC}}$ is a preorder:

$$f \leq_{\text{UC}} f \qquad f \leq_{\text{UC}} g, g \leq_{\text{UC}} h \implies f \leq_{\text{UC}} h$$

universal composition:

$$f \leq_{\text{UC}} g \wedge h \leq_{\text{UC}} k \implies h \circ f \leq_{\text{UC}} k \circ g$$

dummy-adversary completeness — it suffices to simulate the dummy attack ι :

$$\iota_f \approx_{\mathcal{E}} S \circ \iota_g \circ U \text{ for some } S, U \implies f \leq_{\text{UC}} g$$

The last two are UC's pillars — here, each a short categorical proof. (CategoricalCrypto.Abstract)

The metatheory falls out

For any UC-setup, we prove:

$\leq_{\text{UC}}$ is a preorder:

$$f \leq_{\text{UC}} f \quad f \leq_{\text{UC}} g, g \leq_{\text{UC}} h \implies f \leq_{\text{UC}} h$$

universal composition:

$$f \leq_{\text{UC}} g \wedge h \leq_{\text{UC}} k \implies h \circ f \leq_{\text{UC}} k \circ g$$

dummy-adversary completeness — it suffices to simulate the dummy attack ι :

$$\iota_f \approx_{\mathcal{E}} S \circ \iota_g \circ U \text{ for some } S, U \implies f \leq_{\text{UC}} g$$

The last two are UC's pillars — here, each a short categorical proof. (CategoricalCrypto.Abstract)

Instantiate once with concrete machines $\rightarrow$ all theorems for free.